Pie Chart

It's a circular graph having radii divide a circle into sectors proportional in angle to the relative size of the quantities in the category being represented.

Example

This frequency distribution shows the number of pounds of each snack food eaten during the Super Bowl. Construct a pie graph for the data.

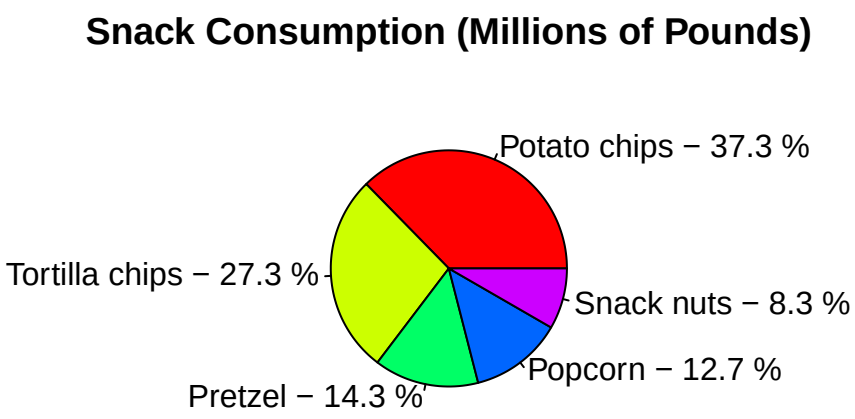
Snack	Potato chips	Tortilla chips	Pretzel	Popcorn	Snack nuts
Pound (in millions)	11.2	8.2	4.3	3.8	2.5

```
# 1. Define the snack data
snacks <- c("Potato chips", "Tortilla chips", "Pretzel", "Popcorn",
            "Snack nuts")
pounds <- c(11.2, 8.2, 4.3, 3.8, 2.5)

# 2. Calculate percentages
percentages <- round(100 * pounds / sum(pounds), 1)

# 3. Create descriptive labels combining name and percentage
pie_labels <- paste(snacks, "-", percentages, "%")

# 4. Generate the pie chart
pie(pounds,
    labels = pie_labels,
    col = rainbow(length(pounds)),
    main = "Snack Consumption (Millions of Pounds)")
```



Bar Chart

A bar chart consists of a set of equal spaced rectangles whose heights are proportional to the frequency of the category /item being considered. The X axis in a bar chart can represent the number of categories. NB Bars are of uniform width and there is equal spacing between them.

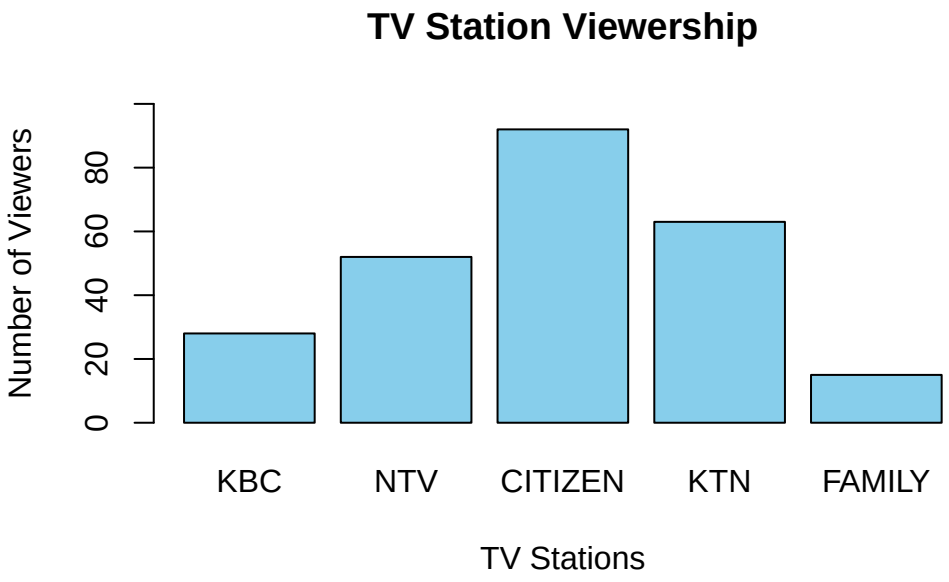
Example

A sample of 250 students was asked to indicate their favourite TV channels and their responses were as follows.

TV Station	KBC	NTV	CITIZEN	KTN	FAMILY
No. of Viewers	28	52	92	63	15

```
# 1. Define the data vectors
stations <- c("KBC", "NTV", "CITIZEN", "KTN", "FAMILY")
viewers  <- c(28, 52, 92, 63, 15)

# 2. Generate the bar chart
barplot(viewers,
        names.arg = stations,
        col = "skyblue",
        main = "TV Station Viewership",
        xlab = "TV Stations",
        ylab = "Number of Viewers",
        ylim = c(0, 100))
```



Pareto Charts

It consist of a set of continuous rectangles where the variable displayed on the horizontal axis is qualitative or categorical and the frequencies are displayed by the heights of vertical bars, which are arranged in order from highest to lowest.

A Pareto chart is used to represent a frequency distribution for a categorical variable.

Example

The table shown here is the average cost per mile for passenger vehicles on state turnpikes. Construct and analyze a Pareto chart for the data.

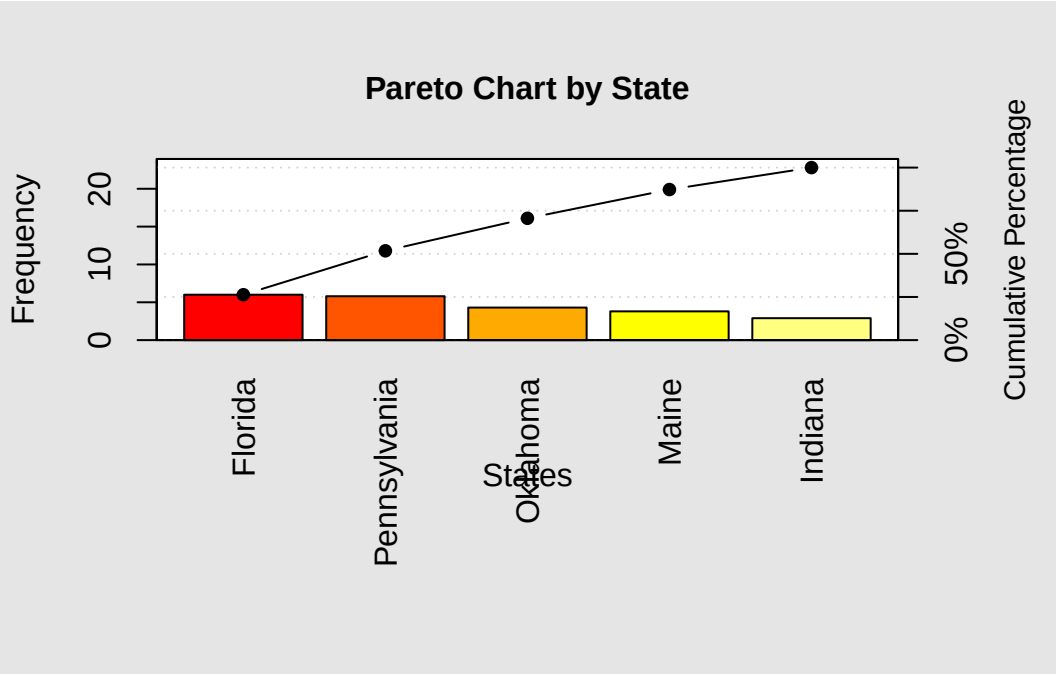
State	Indiana	Oklahoma	Florida	Maine	Pennsylavia
Number	2.9	4.3	6.0	3.8	5.8

```
library(qcc)

# 2. Create the data vector and assign the state names
state_data <- c(2.9, 4.3, 6.0, 3.8, 5.8)

names(state_data) <- c("Indiana", "Oklahoma", "Florida", "Maine",
                       "Pennsylvania")

# 3. Generate the Pareto chart
pareto.chart(state_data,
             main = "Pareto Chart by State",
             col = heat.colors(length(state_data)),
             xlab = "States",
             ylab = "Frequency",
             ylab2 = "Cumulative Percentage")
```




```
Pareto chart analysis for state_data
             Frequency Cum.Freq. Percentage Cum.Percent.
  Florida        6.00000   6.00000   26.31579     26.31579
  Pennsylvania   5.80000  11.80000   25.43860     51.75439
  Oklahoma       4.30000  16.10000   18.85965     70.61404
  Maine          3.80000  19.90000   16.66667     87.28070
  Indiana        2.90000  22.80000   12.71930    100.00000
```

Histogram

It consists of a set of continuous rectangles such that the areas of the rectangles are proportional to the frequency.

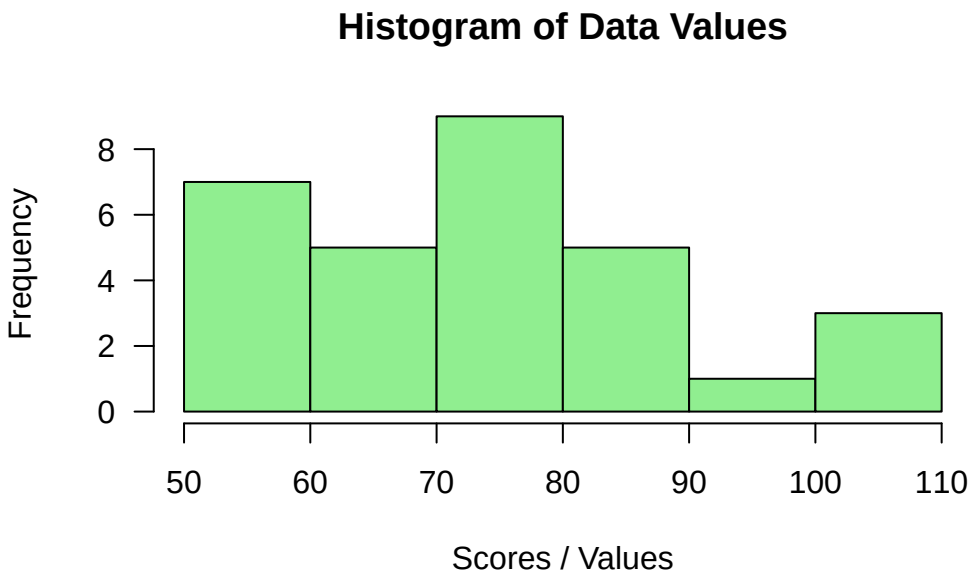
For ungrouped data, the heights of each bar is proportional to frequency.

Example

The number of stories in each of the world's 30 tallest buildings is listed below. Construct a histogram for the data. 88 88 110 88 80 69 102 78 70 55 79 85 80 100 60 90 77 55 75 55 54 60 75 64 105 56 71 70 65 72

```
# 1. Enter the dataset into a vector
data_values <- c(88, 88, 110, 88, 80, 69, 102, 78, 70, 55,
                 79, 85, 80, 100,
             60, 90, 77, 55, 75, 55, 54, 60, 75, 64, 105, 56, 71,
             70, 65, 72)

# 2. Generate the histogram
hist(data_values,
     col = "lightgreen",
     border = "black",
     main = "Histogram of Data Values",
     xlab = "Scores / Values",
     ylab = "Frequency",
     las = 1)
```



Frequency Polygon

It's a plot of frequency against mid points joined with straight line segments between consecutive points.

It can also be obtained by joining the mid point of the tops of the bars in a histogram.

The gaps at both ends are filled by extending to the next lower and upper imaginary classes assuming requency zero.

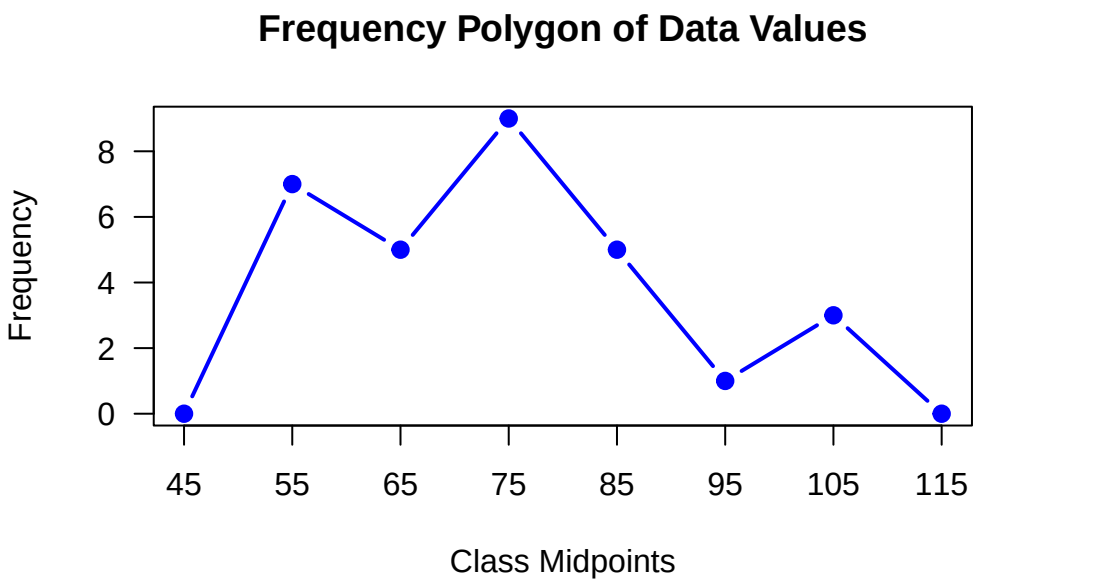
```r
# 1. Enter the dataset into a vector
data_values <- c(88, 88, 110, 88, 80, 69, 102, 78, 70, 55,
                 79, 85, 80, 100, 60, 90, 77, 55, 75, 55,
                 54, 60, 75, 64, 105, 56, 71, 70, 65, 72)

# 2. Compute the underlying histogram counts without plotting it
h <- hist(data_values, plot = FALSE)

# 3. Anchor the polygon to zero at both ends
x_midpoints <- c(h$mids[1] - (h$mids[2] - h$mids[1]),
                 h$mids, h$mids[length(h$mids)] +
                   (h$mids[2] - h$mids[1]))
y_frequencies <- c(0, h$counts, 0)

# 4. Generate the frequency polygon chart
plot(x_midpoints, y_frequencies,
     type = "b",          # "b" plots both points and lines
     pch = 19,            # Solid circle points
     col = "blue",        # Line color
     lwd = 2,             # Line width
     main = "Frequency Polygon of Data Values",
     xlab = "Class Midpoints",
     ylab = "Frequency",
     las = 1,
     xaxt = "n")         # Hide default x-axis to set custom intervals

# 5. Add custom ticks on the X-axis for clarity
axis(1, at = x_midpoints)
```



Cumulative Frequency Curve

It is a plot of cumulative frequency against upper boundaries joined with a smooth curve.

The gap on the lower end is filled by extending to the next lower imaginary class assuming frequency zero.

This graph is useful in estimating median and other measures of location.

```
# 1. Enter the dataset into a vector
data_values <- c(88, 88, 110, 88, 80, 69, 102, 78, 70, 55,
                 79, 85, 80, 100, 60, 90, 77, 55, 75, 55,
                 54, 60, 75, 64, 105, 56, 71, 70, 65, 72)

# 2. Extract the underlying histogram intervals
h <- hist(data_values, plot = FALSE)

# 3. Get the upper class boundaries and add the starting lower boundary
boundaries <- c(h$breaks[1], h$breaks[-1])

# 4. Calculate cumulative frequencies, starting at 0
cumulative_freq <- c(0, cumsum(h$counts))

# 5. Generate the Ogive Curve
plot(boundaries, cumulative_freq,
     type = "b",          # Plots both points and connecting lines
     pch = 19,            # Solid circle points
     col = "darkred",     # Line color
     lwd = 2,             # Line thickness
     main = "Cumulative Frequency Ogive",
     xlab = "Upper Class Boundaries",
     ylab = "Cumulative Frequency",
     las = 1,
     xaxt = "n")         # Hide default x-axis for custom breaks

# 6. Add gridlines and precise axis labels
axis(1, at = boundaries)
grid(nx = NULL, ny = NULL, col = "lightgray", lty = "dotted")
```